Implementação 4 – Sincronização do Metrô

Relatório de atividade – Infraestrutura de Software, CESAR School

Daniel Donaire – ddc@cesar.school
Data: 4 de novembro de 2026

1 Objetivo

O enunciado pede a implementação em C da sincronização de uma estação de metrô: a `struct station` e as quatro funções `station_init`, `station_load_car`, `station_wait_for_car` e `station_on_board`, usando exclusivamente mutex e variáveis de condição da POSIX `pthreads`: no máximo um lock por estação, sem semáforos, sem espera ativa e com embarque simultâneo de vários passageiros em um mesmo carro. Entregamos o código (`sync_car.c`/`sync_car.h`), o `Makefile` que gera o executável `sync_car`, o `evidencias.log` da sessão de desenvolvimento, os dois prints obrigatórios e este relatório; o runner `sync_car-runner.c` foi fornecido pelo professor e vai ser entregue sem nenhuma modificação.

2 Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementar as quatro funções da API em C	**OK**	`./sync_car` imprime `Looks good!` e retorna 0 (log 15:06).
Usar apenas `pthreads` (mutex + variáveis de condição)	**OK**	`grep -n "pthread_"sync_car.c` lista só `mutex_init/lock/unlock` e `cond_init/wait/signal/broadcast` (log 15:06).
Nenhum semáforo ou outra primitiva	**OK**	`grep -ni "sem_"` nos fontes retorna `semaphore: not found` (log 15:06).
No máximo um lock por `station`	**OK**	`grep -c pthread_mutex_t sync_car.h` → `1`; há apenas `pthread_mutex_t lock` na estrutura (log da sessão final).
Sem busy-wait (espera ativa)	**OK**	Todo `while` de espera contém `pthread_cond_wait`; não há `usleep/nanosleep/sched_yield` nos fontes (grep vazio).
Embarque simultâneo (vários passageiros no mesmo carro)	**OK**	Harness auxiliar fora do repositório: 10 threads passam por `wait_for_car` antes do primeiro `on_board` → `SIMULTANEIDADE OK` e `simul exit: 0` (log da sessão final).
`count` = assentos livres: aceita 0, negativo e mais assentos que passageiros	**OK**	Caso limite/inválido: `station_load_car` com 0, −5 e 200 assentos retorna sem travar → `LIMITE OK` (log da sessão final).
`Makefile` gera o executável `sync_car`	**OK**	`make` termina com `cc -o sync_car sync_car.o sync_car-runner.o -pthread` (log 15:06).
Runner do professor entregue sem modificação	**OK**	`sha256sum sync_car-runner.c` confere a integridade do arquivo original (log da sessão final).
`evidencias.log` com data, usuário, diretório e comandos reais	**OK**	Sessões `script(1)` de 14:34 a 22:59, cada uma iniciando com `date; whoami; pwd; uname -a; gcc --version` (seção 6.1).
Dois prints obrigatórios (erro principal + validação final)	**OK**	`prints/erro-early-return.png` e `prints/validacao-final.png` (seção 6.2).
Relatório no formato do guia (seções 1–9)	**OK**	Este documento (`ddc.pdf`).

3 Como Reproduzir

Ambiente testado: Ubuntu 26.04 LTS em WSL2 (Windows 11), `gcc` 15.2.0, GNU Make 4.4.1. O programa não recebe argumentos nem entrada do usuário.

```
make clean
make
./sync_car
```

Saída esperada (trecho): linhas `car entering station with N free seats` / `car departed station with N new passenger(s) (expected N)` e, ao final:

```
Looks good!
```

Para confirmar o código de saída:

```
echo $?
# 0
```

O runner também pode ser chamado por `make test`. A compilação com avisos estritos (`-Wextra -Werror`) é feita por `make check`.

4 Arquitetura (resumida)

Arquivos e responsabilidades:

Arquivo	Responsabilidade
`sync_car.c`	Implementação das quatro funções da estação.
`sync_car.h`	`struct station` (1 mutex, 2 variáveis de condição, 4 contadores) e protótipos.
`sync_car-runner.c`	Programa principal de teste do professor (não modificado).
`Makefile`	Alvos `all`, `check`, `test` e `clean`.
`evidencias.log`	Sessão `script(1)` com comandos, saídas, data, usuário e diretório.
`prints/`	Print do erro principal e da validação final.

Decisões técnicas:

Decisão	**Motivo**	**Consequência**
`count` tratado como assentos livres e clampado (`count > 0 ? count : 0`)	O enunciado define `count` como assentos livres do carro; valores inválidos não podem travar o processo	`load_car(0)` e `load_car(−5)` retornam imediatamente e o caso "mais assentos que passageiros" converge sozinho quando `waiting` chega a 0.
Contador `boarding` + variável de condição `passenger_boarded` (estratégia S2)	S1 não esperava o embarque e o runner abortava com `station_load_car returned early!`	O carro só parte depois de concluído o embarque dos passageiros admitidos (ou quando não há ninguém esperando e ainda sobram assentos); assim a saída do carro passa a ser observável pelo runner.
Uma variável de condição por direção: `car_arrived` (passageiros) e `passenger_boarded` (carro)	Com um único lock, duas direções de espera precisam ser sinalizadas separadamente	`pthread_cond_broadcast` acorda todos os passageiros quando o carro chega; `pthread_cond_signal` devolve o controle ao carro após cada embarque, sem *lost wakeup*.
Condição de saída do carro: `boarding > 0 || (free_seats > 0 && waiting > 0)`	O runner exige que `load_car` não retorne antes de `MIN(passengers_left, free_seats)` embarcarem, nem depois de o embarque terminar	O carro bloqueia enquanto há embarque em andamento ou assento para quem espera, e retorna no máximo 1 s após o último `on_board` (janela de espera do runner).

5 Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

	Nome curto	**Contexto**	**Motivo da troca (se houve)**
S1	Carro publica assentos	Primeira implementação: `station_load_car` define `car_present`/`free_seats`, faz `broadcast` e `return`; `on_board` era vazio	–
S2	Contador de embarque	Correção após a falha do runner: adiciona `boarding`, `passenger_boarded` e o laço de espera na saída do carro	S1 fazia `load_car` retornar imediatamente (o runner chamava `load_car` antes de os passageiros embarcarem) → `Error: station_load_car returned early!`
S3	(não utilizada)	–	–

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Abertura da sessão de evidências (`script -a evidencias.log`) e identificação do ambiente (`date`, `whoami`, `pwd`, `uname -a`, `gcc --version`)	Log iniciado; terminal reaberto em seguida (duas sessões curtas de `exit`)	–	14:34–14:35	`evidencias.log`, blocos 1–3
2	S1	`make clean`, `make`, `make check` e primeira execução do `./sync_car`	**Falha:** `Error: station_load_car returned early!`, `exit code: 1`	`station_load_car` só publicava os assentos e retornava sem esperar o embarque; o runner detectou a saída prematura do carro	14:38–14:39	`evidencias.log`, bloco 14:38:53; print `erro-early-return.png`
3	S2	Reescrita com o contador `boarding` e a variável de condição `passenger_boarded`; `make clean`, `make`, `make check`, `./sync_car`	`Looks good!` e `exit code: 0`	–	15:06	`evidencias.log`, bloco 15:06:02–15:06:58
4	S2	Auditoria dos requisitos por `grep` (campos da estrutura, funções `pthread`, busca por semáforo)	Estrutura com 1 mutex e 2 variáveis de condição; `semaphore: not found`	–	15:06	Mesmo bloco
5	S2	100 execuções seguidas do runner (`for i in 1..100`)	`100 iteracoes OK` (duas tentativas anteriores canceladas com `C` antes de completar)	–	16:14	`evidencias.log`, bloco 16:13:59–16:15:55
6	S2	Harness de simultaneidade em `/tmp/simul.c`: 10 passageiros esperam antes do primeiro `on_board`	`SIMULTANEIDADE OK` e `simul exit: 0`	–	22:58	`evidencias.log`, sessão 22:58–22:59
7	S2	Caso limite/inválido (`count` = 0, −5 e 200), integridade do runner (`sha256sum`), contagem de locks e listagem dos prints	`LIMITE OK`, runner íntegro, `grep -c` = 1, `Looks good!` no print	–	22:58	`evidencias.log`, sessão 22:58–22:59; `prints/`

6 Evidências

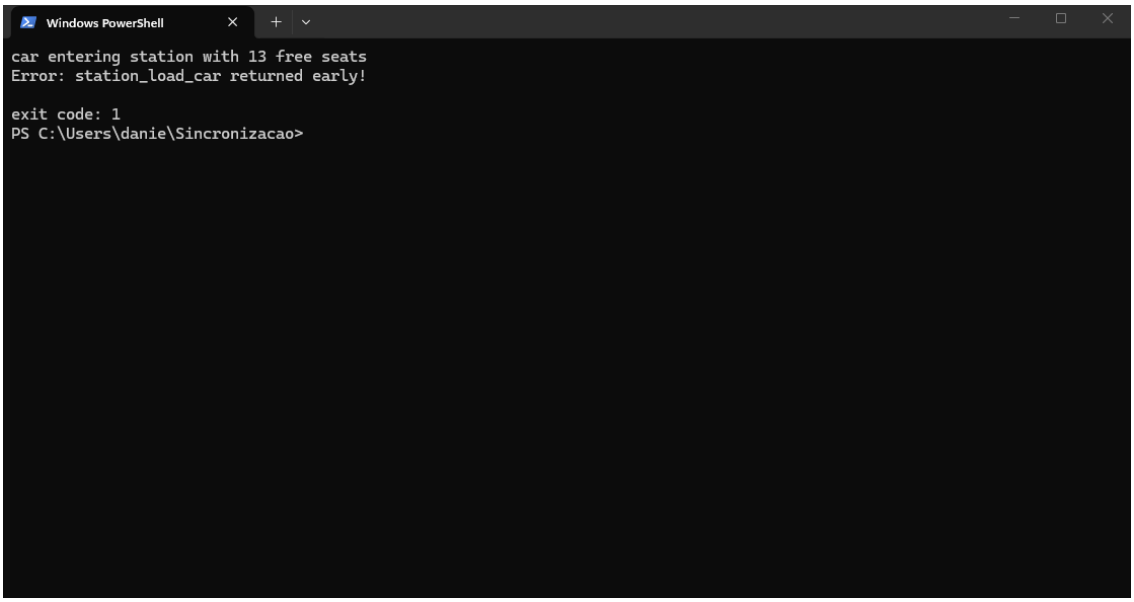
6.1 Log automático

No início de cada sessão de trabalho foi executado `script -a evidencias.log`, que grava automaticamente todos os comandos e saídas do terminal. Toda sessão começa com `date`, `whoami`, `pwd`, `uname -a` e `gcc --version`, de modo que o log carrega sozinho data, usuário, diretório e

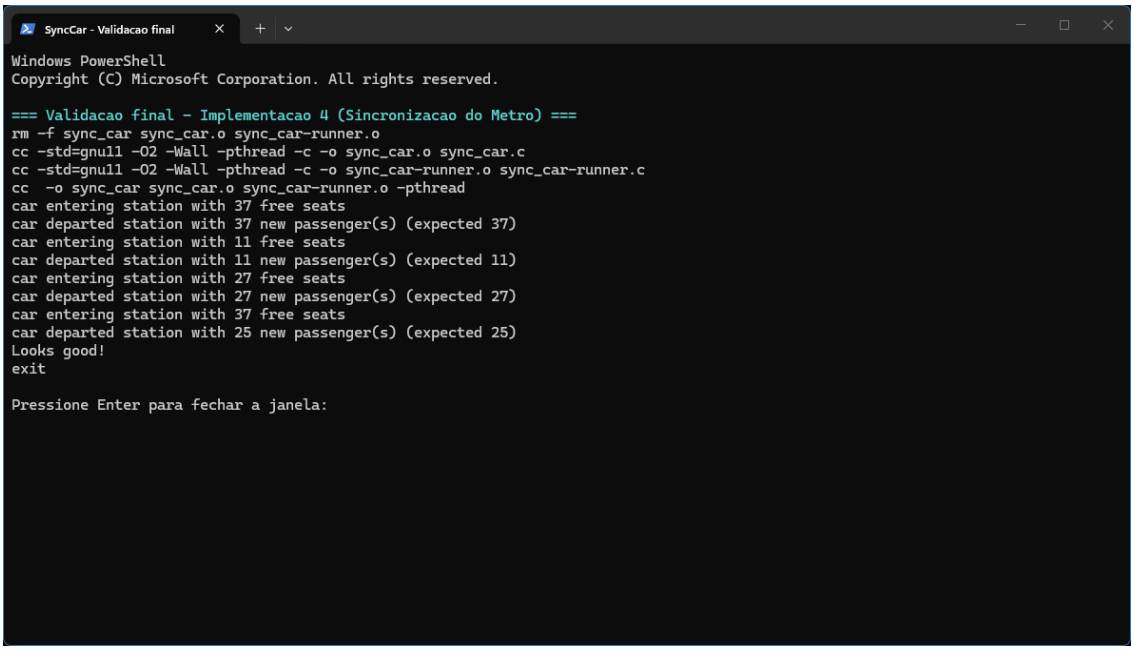
ferramentas. O arquivo `evidencias.log` (enviado dentro do `.tar`) contém 7 sessões entre 14:34 e 22:59 de 04/11/2026, incluindo a falha da estratégia S1, a correção S2, a auditoria de requisitos, o teste de 100 execuções e a sessão final (22:58–22:59), com o caso limite/inválido, a integridade do runner (`sha256sum`), a contagem de locks e o teste de simultaneidade.

6.2 Prints

Erro principal: o momento do bug mais relevante da semana – o runner abortando porque `station_load_car` retornou antes do embarque (estratégia S1):



Validação final: compilação do zero e execução do runner terminando em `Looks good!`:



7 Uso de IA

Onde usei IA (2–3 linhas): utilizei um assistente de IA (OpenCode, com modelo de linguagem) durante toda a atividade: leitura e decomposição das restrições do enunciado e do guia de

relatórios, discussão do desenho da sincronização, rascunhos de código e de testes auxiliares fora do repositório, auditoria dos requisitos por `grep` e montagem da estrutura deste relatório.

O que validei manualmente e como: compilei e executei tudo no meu terminal (`make`, `make check`, `./sync_car`, 100 execuções seguidas, caso limite/inválido e harness de simultaneidade); conferi pessoalmente cada `grep` de auditoria; revi linha a linha o código entregue e este relatório antes de gerar o `ddc.pdf`; decidi sozinho quando trocar de estratégia.

8 Reflexão Final

A principal lição da semana foi confiar no contrato do teste antes de codar: S1 parecia correta, mas esquecia que `station_load_car` é uma operação *síncrona* do carro: ela só pode retornar quando o embarque terminou. Isso causou o bug que mais me travou (`returned early!`) e só foi resolvido quando passei a ler o runner linha a linha e a modelar explicitamente quem espera por quem. A decisão técnica mais importante foi adicionar o contador `boarding`: sem ele, "assentos esgotados" e "embarque em andamento" eram indistinguíveis na condição de saída. Também aprendi a separar as duas direções de espera em variáveis de condição diferentes, o que eliminou a possibilidade de *lost wakeup* sem aumentar o número de locks. Em uma próxima atividade eu faria os testes de concorrência (100 execuções e harness) logo na primeira sessão, em vez de depois da correção, e manteria comentários no código desde o início para reduzir o retrabalho de revisão.

9 Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	✓
Rodei os testes e `evidencias.log` foi gerado	✓
Tenho prints obrigatórios	✓
Testei pelo menos um caso limite e um caso inválido	✓
Preenchi a seção de uso de IA	✓
Revisei o relatório e removi frases genéricas / vazias	✓